PROJECT BASED LEARNING (PBL) REPORT

Chronic Kidney Disease Prognosis Using Various Machine Learning Based Classifiers

Department of Computer Science and Engineering
Symbiosis Institute of Technology, Nagpur, Maharashtra, India
Guide: Dr. Deepak Asudani

ABSTRACT

Chronic Kidney Disease (CKD) is a progressive condition in which kidney function declines gradually, and it is frequently detected only at an advanced stage. Early prognosis can slow progression and reduce the need for dialysis or transplantation. This paper presents a comparative study of seven machine learning classifiers, namely Logistic Regression, K-Nearest Neighbors, Support Vector Machine, Decision Tree, Naive Bayes, Random Forest and Gradient Boosting, for predicting CKD from clinical and laboratory parameters. The UCI Chronic Kidney Disease dataset (400 records, 24 attributes) was pre-processed through missing-value imputation, categorical encoding and feature scaling, followed by feature ranking. Models were evaluated on a held-out test set using accuracy, precision, recall, F1-score and ROC-AUC. Random Forest achieved the best performance with an accuracy of 99.17%, precision of 100%, recall of 98.67% and an AUC of 0.999, followed by Gradient Boosting (98.33%) and SVM (97.50%). Hemoglobin, specific gravity, serum creatinine and albumin were the most influential predictors. The results indicate that ensemble classifiers can serve as a reliable decision-support aid for early CKD detection.

Keywords: *Chronic Kidney Disease, Machine Learning, Classification, Random Forest, Feature Selection, Prognosis*

1. Introduction

Chronic Kidney Disease is a long-term condition defined by abnormalities of kidney structure or function that persist for more than three months [1]. A systematic review estimated the global prevalence of CKD at roughly 11 to 13 percent of the adult population [2], making it a major public health concern, particularly in developing countries where screening is limited. Because early stages are largely asymptomatic, many patients are diagnosed only after substantial and irreversible kidney damage has occurred.

Diagnosis normally depends on blood and urine tests, such as serum creatinine, blood urea, albumin and hemoglobin, that are interpreted by a nephrologist. Machine learning (ML) can learn hidden patterns from such records and classify a patient as CKD or non-CKD within seconds. It can act as a second opinion for doctors and as a screening tool in places where specialists are scarce.

The objective of this work, carried out as a Project Based Learning (PBL) activity, is to build and compare several ML classifiers for CKD prognosis. The main contributions are:

- A complete pipeline covering data cleaning, imputation, encoding, scaling and feature ranking on the UCI CKD dataset.
- Implementation and tuning of seven classifiers under identical experimental conditions.
- A step-by-step explanation of each classifier, with its formulas, a diagram and a worked numerical example.
- A comparison using accuracy, precision, recall, F1-score, confusion matrices and ROC curves.

- Identification of the most important clinical attributes for CKD prediction.

Organisation of the paper. Section 2 reviews related work. Section 3 describes the dataset. Section 4 explains the methodology, including the mathematics behind every model. Section 5 presents and discusses the results, and Section 6 concludes the paper.

2. Literature Review

This section lays the groundwork for the study in three steps. It first explains what chronic kidney disease is and how physicians grade its severity, then outlines the worldwide burden that makes early detection so valuable, and finally surveys how machine-learning researchers have tackled CKD prediction. It closes by pointing out the shortcoming in existing work that this paper sets out to address.

2.1 Clinical Background: Definition and Stages of CKD

Clinical guidelines describe CKD as any abnormality in the structure or function of the kidneys that stays present for longer than three months and has consequences for health [1], [13]. The kidneys clean the blood by passing it through millions of tiny filters, and the overall filtering capacity is summarised by the glomerular filtration rate (GFR). Because GFR cannot be measured directly in routine practice, doctors estimate it (eGFR) from the serum creatinine level together with the patient's age and sex. The KDIGO guideline [13] sorts CKD into five GFR categories, G1 to G5, and divides G3 into G3a and G3b. As Fig. 1 and Table 1 show, the eGFR drops from a normal value above 90 to below 15 as the disease advances, and at the last stage the kidneys can no longer keep the patient alive without dialysis or a transplant.

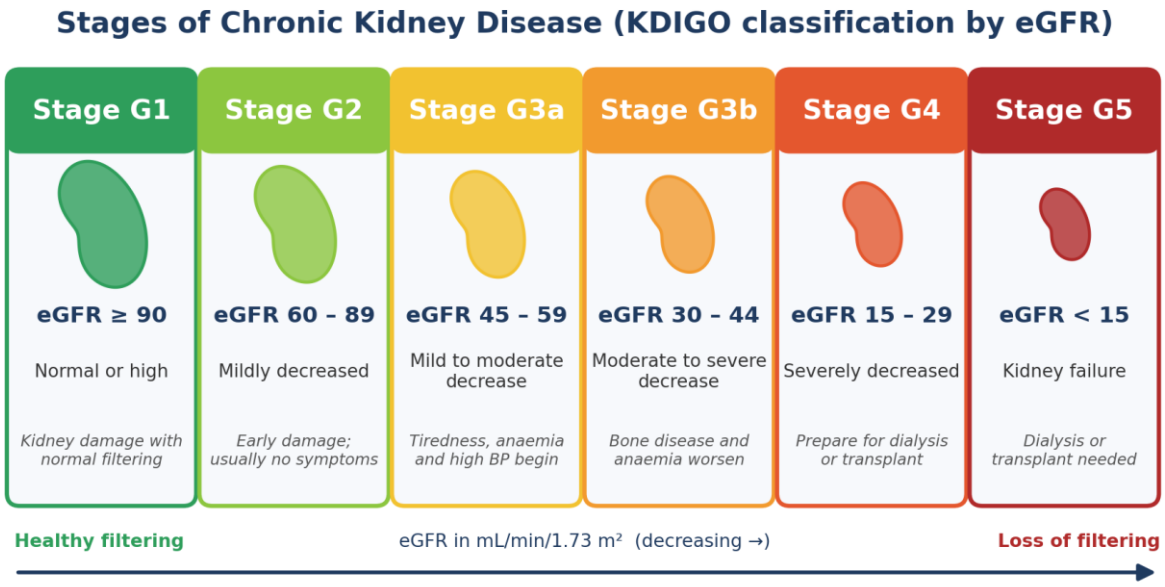


Fig. 1 *Stages of chronic kidney disease according to the KDIGO eGFR categories (illustration prepared by the authors).*

Table 1. Stages of CKD, their typical clinical picture and the usual focus of care (based on [13])

Stage	eGFR	Description	Typical clinical picture	Usual focus of care
G1	≥ 90	Normal or high	Damage is found in urine or scans; symptoms are rare	Treat the underlying cause, control blood pressure and glucose, review yearly
G2	60–89	Mildly decreased	Usually silent and discovered by routine tests	Lifestyle changes, risk-factor control, watch for progression

Stage	eGFR	Description	Typical clinical picture	Usual focus of care
G3a	45–59	Mild to moderate decrease	Tiredness, swelling and rising blood pressure may appear	Check for anaemia and bone-mineral changes, avoid kidney-harming drugs
G3b	30–44	Moderate to severe decrease	Complications become common	Specialist (nephrology) care and active treatment of complications
G4	15–29	Severely decreased	Marked fatigue, fluid retention, poor appetite	Prepare for kidney replacement: education, vascular access, transplant work-up
G5	< 15	Kidney failure	Toxin build-up (uraemia), breathlessness, nausea	Dialysis or kidney transplantation

GFR alone does not tell the whole story. The same guideline therefore adds a second axis, the amount of albumin leaking into the urine, measured as the albumin-to-creatinine ratio (ACR) and grouped into A1 (below 30 mg/g), A2 (30–300 mg/g) and A3 (above 300 mg/g) [13]. Fig. 2 combines the two axes into the well-known risk grid. A patient with an eGFR of 50 and no protein leak sits in the cautious yellow zone, whereas a patient with the same eGFR and heavy albuminuria is already in the red zone. This is one reason why urine albumin and specific gravity are included in the UCI dataset and why they turn up among the strongest predictors in Section 5.

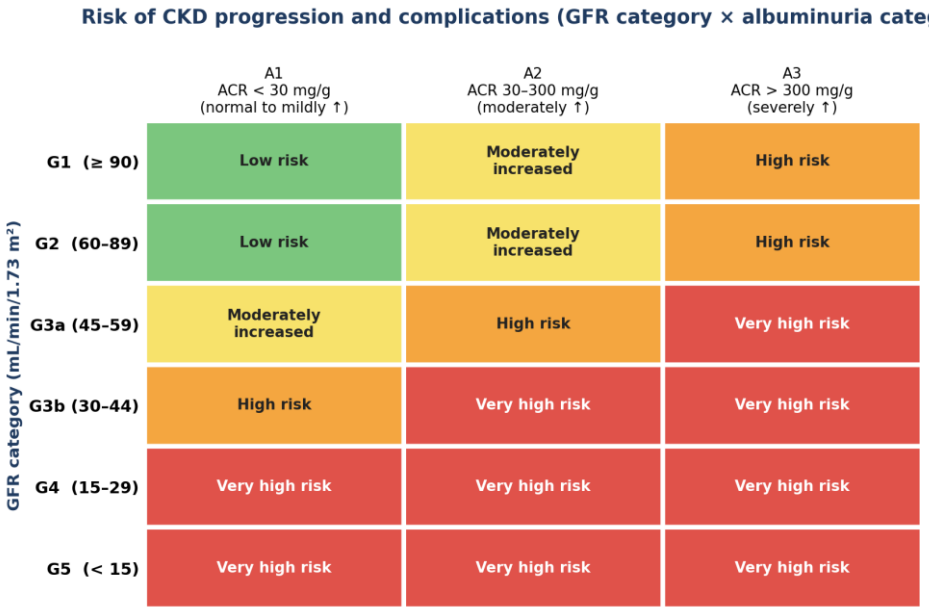


Fig. 2 *Risk of progression and complications for each combination of GFR and albuminuria category [13].*

WORKED EXAMPLE: HOW eGFR IS ESTIMATED FROM CREATININE

The 2021 CKD-EPI creatinine equation [15] is **eGFR = 142 × min(Scr/κ, 1)^α × max(Scr/κ, 1)^−1.200 × 0.9938^Age × 1.012 (if female)**, where Scr is serum creatinine in mg/dl, κ = 0.9 for men and 0.7 for women, and α = −0.302 for men and −0.241 for women.

Patient A: man, 50 years, Scr = 1.0 mg/dl gives eGFR ≈ **92**, which is in the G1 range. It would count as CKD only if another marker of damage, such as albuminuria, were also present.

Patient B: the same man with Scr = 2.5 mg/dl gives eGFR ≈ **31**, which is stage **G3b**. A rise of only 1.5 mg/dl in creatinine therefore moves the patient across three stages, which explains why serum creatinine is one of the top predictors in CKD models.

2.2 Global Burden and the Case for Early Detection

Hill et al. [2] pooled the results of many population studies and arrived at a global prevalence close to 13 percent, with the majority of affected people still in the early stages. A later analysis for the Global Burden of Disease Study 2017 [14] counted nearly 700 million people living with CKD and linked the condition to roughly 1.2 million deaths in that year. Two features of the disease make these figures worrying. First, the early stages produce few or no symptoms, so many patients learn about their condition only when the filtering capacity is already badly reduced. Second, kidney replacement therapy is expensive and hard to reach in low-resource regions. Cheap, quick screening that flags high-risk patients from ordinary laboratory values would help clinicians start treatment while the damage can still be slowed, and this need is what motivates the machine-learning work reviewed next.

2.3 Evolution of Machine-Learning Research on CKD

Interest in data-driven CKD diagnosis grew quickly after the UCI repository released a labelled CKD dataset [11]. Most of the resulting studies can be placed in one of four themes: data preparation and feature selection, classical classifiers, ensemble methods and neural networks. Fig. 3 maps these themes to the key references discussed in the rest of this section.

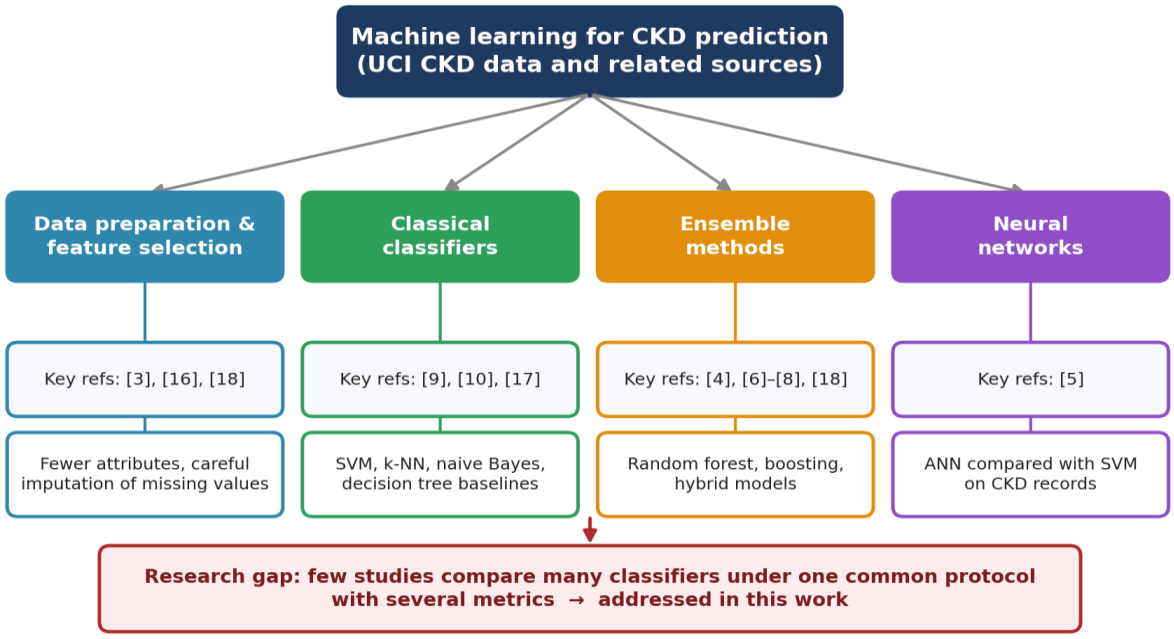


Fig. 3 *Research themes in CKD prediction and the gap addressed by this work.*

2.4 Feature Selection and Data Preparation

A first group of studies asked which of the 24 attributes are really needed. Salekin and Stankovic [3] compared k-nearest neighbours, random forest and a neural network after reducing the attribute set, and showed that a compact subset could keep the predictive quality of the full set. Polat et al. [16] paired a support vector machine with both filter-style and wrapper-style selection methods and reached the same conclusion: fewer tests can be enough, which matters because every extra laboratory test costs the patient money and time. Qin et al. [18] focused on a different weakness, the many empty cells in the UCI records. Their work illustrates that the way missing values are filled can change the final accuracy as much as the choice of classifier. This observation is the reason we document our median and mode imputation in Section 4.2.

2.5 Classical and Neural Classifiers

Single-model classifiers were the first to be tried. Tazin et al. [17] tested the decision tree, naive Bayes, SVM and k-NN, and reported that the decision tree gave the highest accuracy of the models they compared.

Almansour et al. [5] put an artificial neural network against an SVM on CKD data and found that both reached strong results once the data had been cleaned and scaled, with the network slightly ahead. The SVM of Cortes and Vapnik [9] and the nearest-neighbour rule of Cover and Hart [10] continue to serve as standard baselines because they are simple, well understood and easy to reproduce. Their drawback is sensitivity to feature scale, which is why we standardise the inputs for these models.

2.6 Ensemble Methods

Ensembles combine many weak learners into one stronger predictor. Subasi et al. [4] compared random forest with several other learners on CKD data and found the forest to be the most effective. The method, introduced by Breiman [6], grows many decision trees on random samples of the records and attributes and takes a majority vote, which lowers variance. Boosting follows a different idea: trees are added one after another, each correcting the errors of the previous ones, as formalised by Friedman [7] and engineered for speed by Chen and Guestrin [8]. Qin et al. [18] went one step further and built an integrated model that joins logistic regression with a random forest. Ensembles suit CKD records well because the data mix numeric and categorical fields, contain gaps and outliers, and do not need strict scaling, and because tree-based models give an importance score for every attribute.

Table 2. Summary of representative studies on CKD prediction and what each contributes to this work

Study	Focus	Techniques	Relevance to this work
Salekin and Stankovic [3]	Attribute reduction before classification	k-NN, random forest, neural network	Motivates the feature ranking carried out in Section 4.3
Polat et al. [16]	Filter and wrapper feature selection	SVM	Shows that a shorter attribute list can be enough for diagnosis
Tazin et al. [17]	Comparison of basic classifiers	Decision tree, naive Bayes, SVM, k-NN	Supports using tree and distance-based models as baselines
Subasi et al. [4]	Diagnosis of CKD with ensembles	Random forest against other learners	Random forest is a strong candidate model
Almansour et al. [5]	Network versus margin classifier	Artificial neural network, SVM	Stresses the value of cleaning and scaling the data
Qin et al. [18]	Missing-value handling and hybrid model	Logistic regression with random forest	Shows that imputation choices affect results

2.7 Critical Observations and Research Gap

Reading these papers side by side reveals several points that shape the design of our experiments.

> **KEY OBSERVATIONS FROM THE LITERATURE**
>
> **1. Very high scores on a small dataset.** Many studies report near-perfect results on only 400 records from a single hospital. Such numbers may be optimistic and may not carry over to other populations.
>
> **2. Unequal experimental conditions.** Studies differ in how they impute, scale and split the data, so their scores cannot be compared fairly.
>
> **3. Accuracy hides errors.** About 37.5 percent of the UCI records are non-CKD. A model can look accurate and still miss patients, and a missed CKD case is more harmful than a false alarm, so recall and ROC-AUC are needed as well.
>
> **4. Explanations matter to doctors.** Clinicians trust a model more when they can see which laboratory values drive a decision, so attribute importance should be reported.

Table 3. Gaps found in earlier work and how this paper responds

Gap	What earlier work shows	Response in this paper
Limited model coverage	Most papers test two to four classifiers	Seven classifiers from four families are compared
Accuracy as the only metric	Little information on missed patients	Accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC are reported
Unclear pre-processing	Different imputation and scaling in each study	One documented pipeline applied identically to every model
Weak interpretability	Importance of attributes rarely discussed	Random-forest importance and correlation analysis identify key tests
Hard-to-follow methods	Mathematics of models seldom explained	Each classifier is explained with formulas, a diagram and a worked example

In short, earlier research has proved that machine learning can separate CKD from non-CKD patients with high accuracy, but it has not given a clear, like-for-like comparison of the main classifier families with a full set of metrics. The following sections fill this gap using the UCI dataset and a single, transparent experimental protocol.

3. Dataset Description

3.1 Source and Overview

The dataset used in this study is the Chronic Kidney Disease dataset from the UCI Machine Learning Repository [11]. It was collected from patients of a hospital in Tamil Nadu, India, over a period of about two months, and donated to the repository in 2015. It is one of the most widely used public benchmarks for CKD prediction, which allows our results to be compared with earlier work.

The dataset contains 400 instances (patient records) and 24 predictor attributes plus one target attribute called class. Of the 24 predictors, 11 are numeric and 13 are nominal or ordinal. The target is binary: 250 patients (62.5%) are labelled ckd and 150 patients (37.5%) are labelled notckd, so the classes are moderately imbalanced (Fig. 4a).

3.2 Other Datasets Used in CKD Research

For context, Table 4 lists other data sources that researchers use for CKD modelling. They differ from the UCI data in size, setting and purpose, which is why the UCI set remains the common benchmark for comparing classifiers.

Table 4. Other data sources used in CKD machine learning research

Dataset	Description and typical use
UCI CKD	Benchmark used in this work: 400 patients from one hospital, 25 clinical attributes including age, blood pressure, specific gravity, albumin, blood urea and serum creatinine.
NHANES	Survey-based data that combines interviews with physical and laboratory examinations. Subsets are used to build models from demographic, lifestyle and laboratory data (such as the urine albumin-to-creatinine ratio) in the general population.
MIMIC-IV extracts	Electronic health records of critical-care patients. Used for time-series forecasting of CKD progression and acute complications from vital signs, laboratory results and medication records.
Health-claims databases	Billing and insurance records with very large sample sizes and multi-year follow-up (for example 24 months). Used to find early comorbidities and medicines that predict CKD before clinical diagnosis.
CRIC study	Longitudinal research cohort that follows thousands of adults with mild-to-moderate CKD for many years. Designed to study risk factors for progression and related heart disease rather than a simple yes/no diagnosis.

3.3 Attribute Description

The attributes cover blood tests, urine tests and the medical history of the patient. Table 5 gives the full data dictionary. Specific gravity, albumin and sugar are recorded as small ordered integer levels and are treated as ordinal features.

Table 5. Data dictionary of the UCI CKD dataset

#	Attribute	Description	Type	Values / unit
1	age	Age of the patient	Numeric	years
2	bp	Blood pressure (diastolic)	Numeric	mm/Hg
3	sg	Specific gravity of urine (concentration ability)	Ordinal	1.005, 1.010, 1.015, 1.020, 1.025
4	al	Albumin in urine (protein leakage)	Ordinal	0 to 5
5	su	Sugar in urine	Ordinal	0 to 5
6	rbc	Red blood cells in urine	Nominal	normal / abnormal
7	pc	Pus cells in urine	Nominal	normal / abnormal
8	pcc	Pus cell clumps	Nominal	present / not present
9	ba	Bacteria in urine	Nominal	present / not present
10	bgr	Random blood glucose	Numeric	mgs/dl
11	bu	Blood urea	Numeric	mgs/dl
12	sc	Serum creatinine	Numeric	mgs/dl
13	sod	Sodium	Numeric	mEq/L
14	pot	Potassium	Numeric	mEq/L
15	hemo	Hemoglobin	Numeric	gms
16	pcv	Packed cell volume	Numeric	%
17	wc	White blood cell count	Numeric	cells/cumm
18	rc	Red blood cell count	Numeric	millions/cmm

#	Attribute	Description	Type	Values / unit
19	htn	Hypertension	Nominal	yes / no
20	dm	Diabetes mellitus	Nominal	yes / no
21	cad	Coronary artery disease	Nominal	yes / no
22	appet	Appetite	Nominal	good / poor
23	pe	Pedal edema (ankle swelling)	Nominal	yes / no
24	ane	Anemia	Nominal	yes / no
25	class	Target: diagnosis	Nominal	ckd / notckd

3.4 Clinical Relevance of Key Attributes

The most important attributes are linked to how the kidney works. Table 6 explains what each one measures and why it matters.

Table 6. Key clinical features in CKD datasets

Feature	Category	Description	Clinical importance
Serum creatinine	Blood test	Waste product from muscle metabolism.	Primary indicator of renal function; used to calculate the estimated Glomerular Filtration Rate (eGFR).
Blood urea nitrogen (BUN)	Blood test	Amount of nitrogen in blood from urea.	Elevated levels indicate impaired kidney clearance.
Hemoglobin	Blood test	Protein in red blood cells that carries oxygen.	CKD often reduces erythropoietin production, leading to anemia.
Albumin	Urine test	Protein present in urine.	High levels (albuminuria) signify damage to the kidney filters (glomeruli).
Specific gravity	Urine test	Concentration of solutes in the urine.	Measures the kidney's ability to concentrate urine, which drops as disease progresses.
Hypertension (HTN)	Comorbidity	History of high blood pressure.	Both a leading cause and a common complication of CKD.
Diabetes mellitus (DM)	Comorbidity	History of diabetes.	Diabetic nephropathy is a leading cause of chronic kidney failure worldwide.

In addition, sodium and potassium describe the electrolyte balance, which is disturbed in advanced kidney disease, and packed cell volume and RBC count fall together with hemoglobin because of anemia.

3.5 Data Quality Issues

The raw dataset is not clean, and this is what makes pre-processing important. Many values are missing: red blood cells (38%), white blood cell count (26.5%), RBC count (32.5%), potassium (22%), sodium (21.8%), packed cell volume (17.8%), hemoglobin (13%) and random glucose (11%), as shown in Fig. 4b. Some nominal fields also contain stray tab characters or spelling variations (for example “ckd\t” or “yes” written inconsistently) that must be standardised before encoding. Only a minority of records are fully complete.

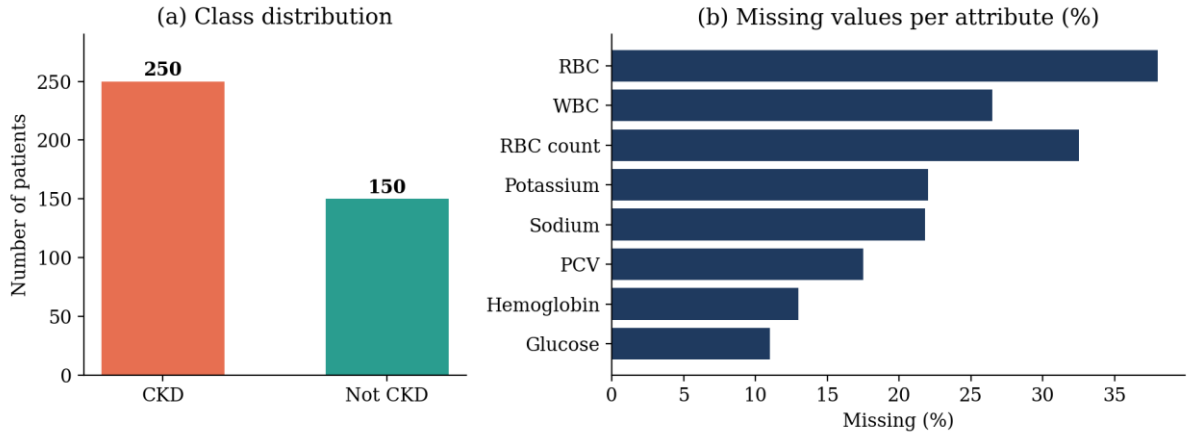


Fig. 4 *(a) Class distribution and (b) attributes with the most missing values.*

4. Methodology

4.1 Overall Workflow

The overall workflow followed in this study is shown in Fig. 5. Each stage is described in the sections below.

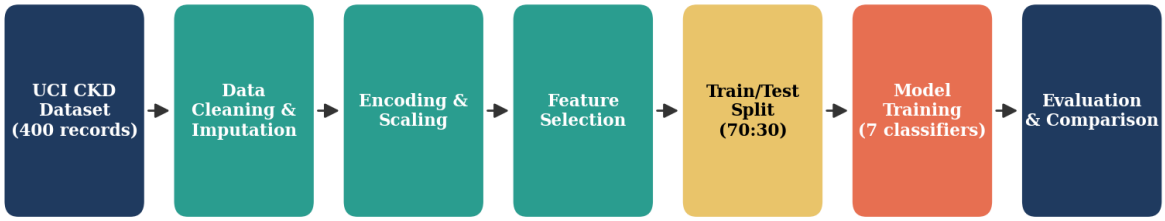


Fig. 5 *Proposed workflow for CKD prognosis.*

4.2 Data Pre-processing

Pre-processing turns the raw hospital records into clean numbers that a model can use. It was done in four steps:

1. **Cleaning.** Stray characters and tab spaces in categorical fields were removed and labels were standardised (for example “ckd\t” became “ckd”).
2. **Missing values.** Numeric attributes were filled with the **median** of the known values and nominal attributes with the **mode** (the most frequent category).
3. **Encoding.** Nominal attributes were converted to numbers using label encoding (for example normal = 0, abnormal = 1).
4. **Scaling.** Standard scaling was applied for distance- and margin-based models (KNN, SVM, Logistic Regression).

Formulas

For an attribute with known values $x_1, \ldots, x_n$, the median imputation and standard scaling are:

$$x_{\text{missing}} \leftarrow \text{median}(x_1, x_2, \ldots, x_n) \tag{1}$$

$$z = \frac{x - \mu}{\sigma} \tag{2}$$

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2} \tag{3}$$

Here x is one raw value, μ is the mean, σ is the standard deviation and z is the scaled value (the "z-score"). After scaling, every attribute has a mean of 0 and a standard deviation of 1, so no single attribute dominates the distance calculations.

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)

Median vs. mean. Suppose five patients have serum creatinine values 0.8, 1.0, 1.2, 4.4 and 7.2 mg/dl and a sixth value is missing. Sorting gives the middle value **1.2** (median). The mean would be 2.92, which is pulled up by the two very high values, so the median is a safer fill value.

Scaling. If hemoglobin has mean μ = 12.5 and standard deviation σ = 2.9, a patient with 8.5 g/dl gets z = (8.5 − 12.5) / 2.9 = **−1.38**, meaning 1.38 standard deviations below average.

4.3 Feature Selection

Features were ranked using Random Forest importance scores and cross-checked with correlation analysis. The top ten attributes are shown in Fig. 6, with hemoglobin, specific gravity, serum creatinine and albumin contributing the most.

How the importance score is calculated

1. Every time a tree splits on feature f, the impurity of the data falls by an amount Δi (see the Gini index in Section 4.4.4).
2. Weight that drop by the share of patients reaching the node, N_t / N, so that splits near the top of the tree count more.
3. Add up the weighted drops for all splits that use feature f, and average over all trees.
4. Divide by the total over all features so that the importances add up to 1.

$$\mathrm{Imp}(f) = \frac{1}{B}\sum_{b=1}^{B}\sum_{t}\frac{N_t}{N}\Delta i(t) \tag{4}$$

B is the number of trees, t runs over the nodes of tree b that split on feature f, N_t is the number of patients at node t and N is the total number of training patients.

The correlation check uses Pearson's coefficient between two attributes x and y. A value near +1 or −1 means that the two move together, so one of them adds little new information:

$$\rho = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}} \tag{5}$$

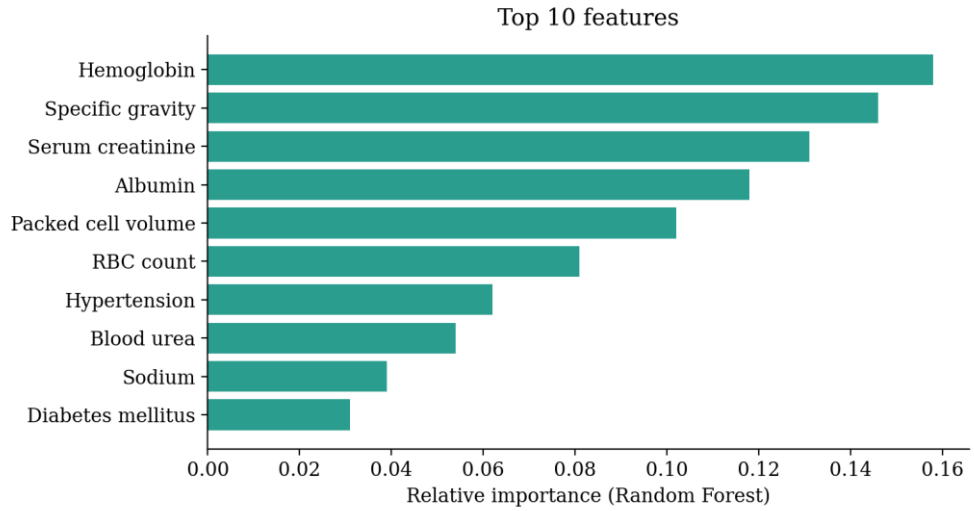


Fig. 6 *Top ten features ranked by importance.*

4.4 Model Description

Seven supervised classifiers with different learning principles were selected: linear, instance-based, margin-based, tree-based, probabilistic and ensemble (Fig. 7). All models receive the same pre-processed feature set and the same train/test split, so differences in performance come from the algorithms themselves. Table 7 compares them at a glance, and Table 8 (Section 4.5) lists the hyperparameters used.

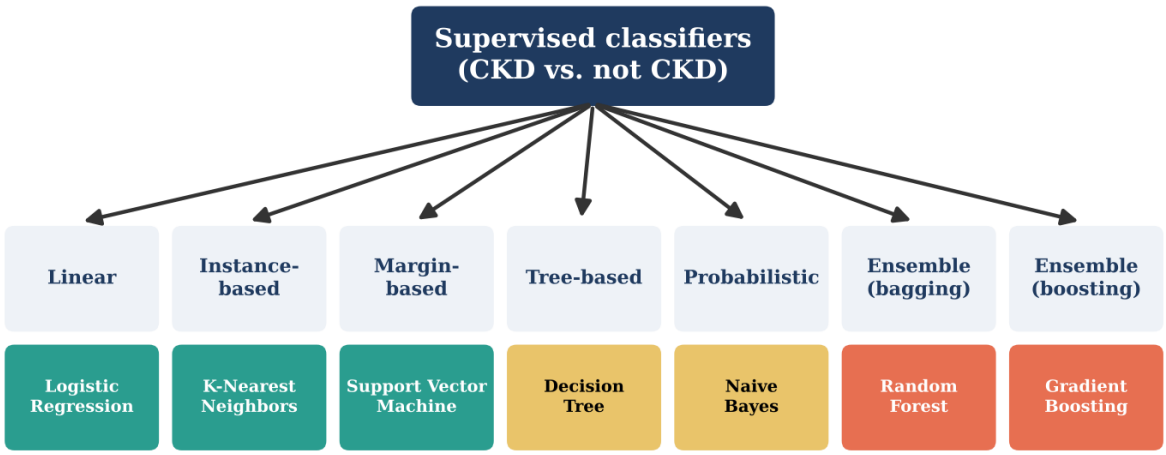


Fig. 7 *The seven classifiers grouped by learning principle.*

Table 7. Quick comparison of the seven classifiers

Classifier	Core idea	Needs scaling	Easy to explain	Main weakness
Logistic Regression	Weighted sum, then sigmoid	Yes	Very	Only straight-line boundaries
KNN	Vote of the k closest patients	Yes	Very	Slow and noise-sensitive
SVM	Widest-margin boundary	Yes	Medium	Needs tuning of C and gamma
Decision Tree	Chain of if-then questions	No	Very	Unstable, overfits
Naive Bayes	Bayes rule with independence	No	Medium	Correlated features break the assumption
Random Forest	Many trees vote	No	Low	Needs more memory
Gradient Boosting	Trees fix each other's errors	No	Low	Sequential and sensitive to settings

4.4.1 Logistic Regression (LR)

> **IN SIMPLE WORDS**
> Give each lab value a weight, add everything into one score, and turn the score into a probability between 0 and 1.

Logistic Regression is a linear model for binary classification. It computes a weighted sum of the input features and passes it through the sigmoid function to obtain the probability that a patient has CKD. A patient is classified as CKD if this probability is 0.5 or more. The weights are learned by minimising the log-loss with L2 regularisation to avoid overfitting.

Step by step

1. Scale every feature (standard scaling, Section 4.2).
2. Compute the score z, a weighted sum of the features plus a bias term b.
3. Pass z through the sigmoid function. Large positive z gives a probability near 1, large negative z gives a probability near 0 (Fig. 8).
4. If P(CKD | x) ≥ 0.5, predict CKD; otherwise predict not CKD.
5. During training, adjust w and b to minimise the loss J below. A solver (lbfgs) repeats this until the loss stops improving.

Formulas

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_p x_p + b = \mathbf{w} \cdot \mathbf{x} + b \tag{6}$$

$$P(\text{CKD} \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{7}$$

$$\ln\left(\frac{P}{1-P}\right) = z \tag{8}$$

$$J(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} [-y_i \ln p_i - (1 - y_i)\ln(1 - p_i)] \tag{9}$$

w is the vector of weights, x the feature vector, b the bias, y_i the true label (1 = CKD) and p_i the predicted probability. The first term of J keeps the weights small (L2 regularisation) and the second term is the log-loss, which punishes confident wrong predictions. C controls the balance between them. Equation (8) shows why LR is easy to interpret: the log-odds change by exactly w_j when feature j rises by one unit.

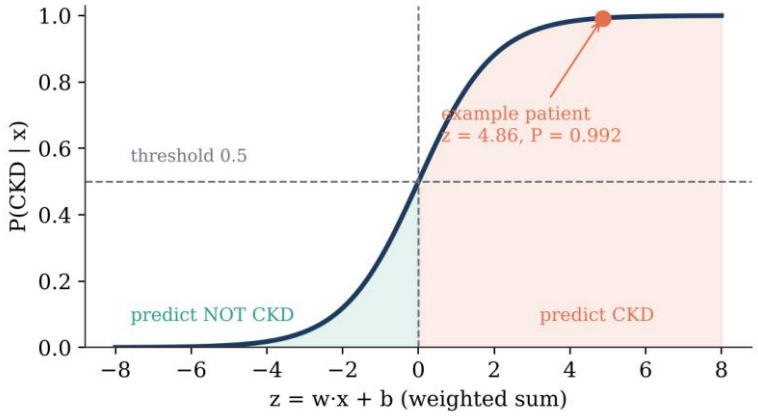


Fig. 8 *The sigmoid function turns the score z into a probability. The marked point is the worked example below.*

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)

Assume a fitted model with weights for hemoglobin, serum creatinine and specific gravity (all standardised) and bias b = 0.3. A patient has the values below.

Feature	Value x	Weight w	Contribution w·x
Hemoglobin	−1.5	−1.2	+1.80
Serum creatinine	+2.0	+0.9	+1.80
Specific gravity	−1.2	−0.8	+0.96
Bias b			+0.30
Score z			**4.86**

Probability: P = 1 / (1 + exp(−4.86)) = **0.992**, which is above 0.5, so the patient is classified as **CKD**. Because exp(−1.2) = 0.30, every one-standard-deviation rise in hemoglobin multiplies the odds of CKD by about 0.30.

Strengths and limitations: Simple, fast and easy to interpret, since each weight shows the effect of a feature. It cannot capture non-linear relationships between attributes.

4.4.2 K-Nearest Neighbors (KNN) [10]

IN SIMPLE WORDS

A new patient is compared with every stored patient. The k most similar ones vote, and the majority label wins.

KNN is an instance-based method that stores the training data and delays learning until prediction. For a new patient, it finds the k most similar training records using Euclidean distance and assigns the majority class among them. Here k = 5 was chosen by cross-validation. Because it relies on distance, scaling is essential.

Step by step

1. Scale all features so that each contributes equally to the distance.
2. Measure the Euclidean distance from the new patient to every training patient.
3. Sort the distances and keep the k = 5 smallest.

4. Count the classes of these 5 neighbours; the majority is the prediction (Fig. 9).

Formulas

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2} \tag{10}$$

$$\hat{y}(\mathbf{x}) = \operatorname{argmax}_c \sum_{i \in N_k(\mathbf{x})} \mathbf{1}(y_i = c) \tag{11}$$

d(x, y) is the distance between two patients with p features each. $N_k(x)$ is the set of the k nearest training patients, y_i is the label of neighbour i, and $1(\cdot)$ equals 1 when the statement is true and 0 otherwise. In words: for each class c, count how many of the k neighbours belong to it, and pick the class with the highest count.

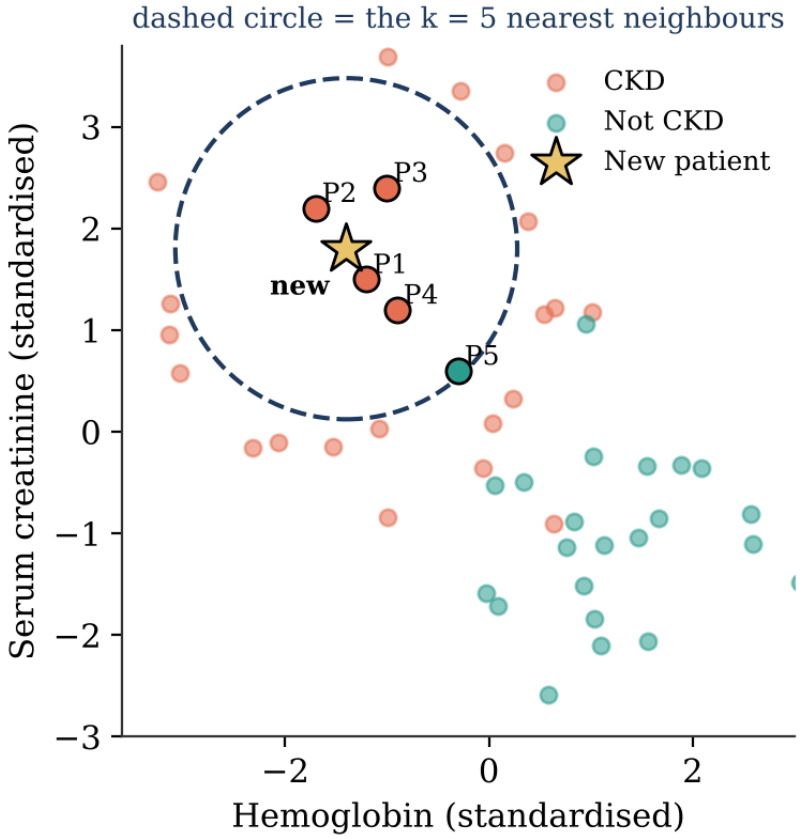


Fig. 9 *KNN with k = 5. The new patient (star) is assigned the majority class of the five closest records.*

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)

A new patient has standardised hemoglobin −1.4 and serum creatinine 1.8, so q = (−1.4, 1.8). Distance to P1 = (−1.2, 1.5): $d = \sqrt{[(-1.4 + 1.2)^2 + (1.8 - 1.5)^2]} = \sqrt{(0.04 + 0.09)} = \mathbf{0.361}$. The five closest patients are:

Neighbour	Coordinates	Class	Distance to q	Rank
P1	(−1.2, 1.5)	CKD	0.361	1
P2	(−1.7, 2.2)	CKD	0.500	2
P3	(−1.0, 2.4)	CKD	0.721	3
P4	(−0.9, 1.2)	CKD	0.781	4
P5	(−0.3, 0.6)	Not CKD	1.628	5

Votes: 4 for CKD and 1 for not CKD, so the patient is classified as **CKD**.

Strengths and limitations: No training phase and easy to understand. It is slow on large data and sensitive to noise, irrelevant features and imputed values, which explains its lower score here.

4.4.3 Support Vector Machine (SVM) [9]

IN SIMPLE WORDS

Draw the boundary that keeps the widest possible empty street between the two classes. Only the points closest to the street matter.

SVM looks for the hyperplane that separates the two classes with the largest possible margin. The training points closest to this boundary are called support vectors. With the RBF kernel, the data is implicitly mapped to a higher-dimensional space, which lets SVM draw non-linear boundaries. The parameter C controls the trade-off between a wide margin and misclassification, and gamma controls how far the influence of one sample reaches.

Step by step

1. Scale all features.
2. Look for a boundary $w \cdot x + b = 0$ that separates CKD from not CKD. Many such lines exist.
3. Choose the one with the largest margin, that is, the largest distance to the nearest points of each class (Fig. 10a). Those nearest points are the support vectors.
4. Real data overlaps, so allow a few points inside the margin (slack ξ). The parameter C sets how strongly such mistakes are punished.
5. If the classes cannot be separated by a straight line, apply the RBF kernel, which measures similarity between patients and bends the boundary (Fig. 10b).
6. To classify a new patient, check on which side of the boundary it falls.

Formulas

$$\text{margin} = \frac{2}{\|\mathbf{w}\|} \tag{12}$$

$$\min \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{subject to} \quad y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i \tag{13}$$

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right) \tag{14}$$

$$f(\mathbf{x}) = \text{sign}\left(\sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b\right) \tag{15}$$

Equation (13) says: make the margin as wide as possible (small ‖w‖) while keeping the slack ξ small. y_i is +1 or −1. The kernel K gives a value close to 1 for similar patients and close to 0 for very different ones; γ controls how quickly the similarity falls with distance. In the decision function, α_i is non-zero only for support vectors.

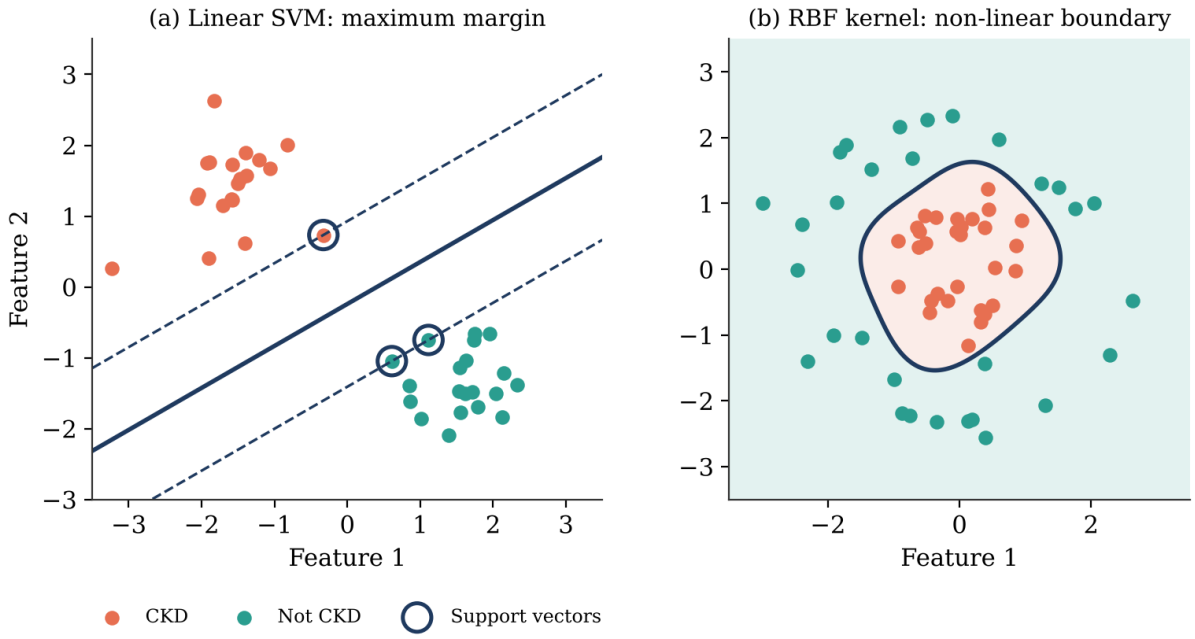


Fig. 10 *(a) A linear SVM keeps the widest margin between the classes; circled points are support vectors. (b) The RBF kernel lets the boundary curve around the data (synthetic illustration).*

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)

Take two patients x = (1, 2) and x′ = (2, 0) and γ = 0.5. The squared distance is $\|x - x'\|^2 = (1-2)^2 + (2-0)^2 = 5$. Then $K = \exp(-0.5 \times 5) = \exp(-2.5) = \mathbf{0.082}$, a low similarity. A closer pair such as (1, 2) and (1.2, 2.1) gives $\|\cdot\|^2 = 0.05$ and $K = \exp(-0.025) = \mathbf{0.975}$, a very high similarity.

Strengths and limitations: Effective in high-dimensional spaces and with small datasets. It needs feature scaling, careful tuning of C and gamma, and does not directly output probabilities.

4.4.4 Decision Tree (DT)

IN SIMPLE WORDS

Play “20 questions” with the lab values. At each step ask the question that separates CKD and non-CKD patients best.

A decision tree splits the data repeatedly using the attribute and threshold that give the purest child nodes, measured by the Gini index. The result is a tree of if-then rules, for example: if hemoglobin is below a threshold and specific gravity is low, then predict CKD. The depth of the tree was limited to reduce overfitting.

Step by step

1. Start with all training patients in the root node and compute its Gini impurity.
2. For every feature and every possible threshold, split the node into a left and a right child.
3. Compute the weighted Gini impurity of the two children and calculate the gain (how much impurity fell).
4. Keep the split with the largest gain and repeat inside each child.

5. Stop at the maximum depth (5) or when a node is pure. A new patient simply follows the questions down to a leaf, and the leaf's majority class is the prediction.

Formulas

$$\text{Gini} = 1 - \sum_k p_k^2 \tag{16}$$

$$\text{Gain} = \text{Gini}_{\text{parent}} - \sum_{\text{child}} \frac{n_{\text{child}}}{n} \text{Gini}_{\text{child}} \tag{17}$$

p_k is the share of class k inside the node. A pure node (all CKD or all not CKD) has Gini = 0, and a 50:50 node has the maximum value of 0.5 (Fig. 11a). n is the number of patients in the parent node, and each child is weighted by its share of those patients.

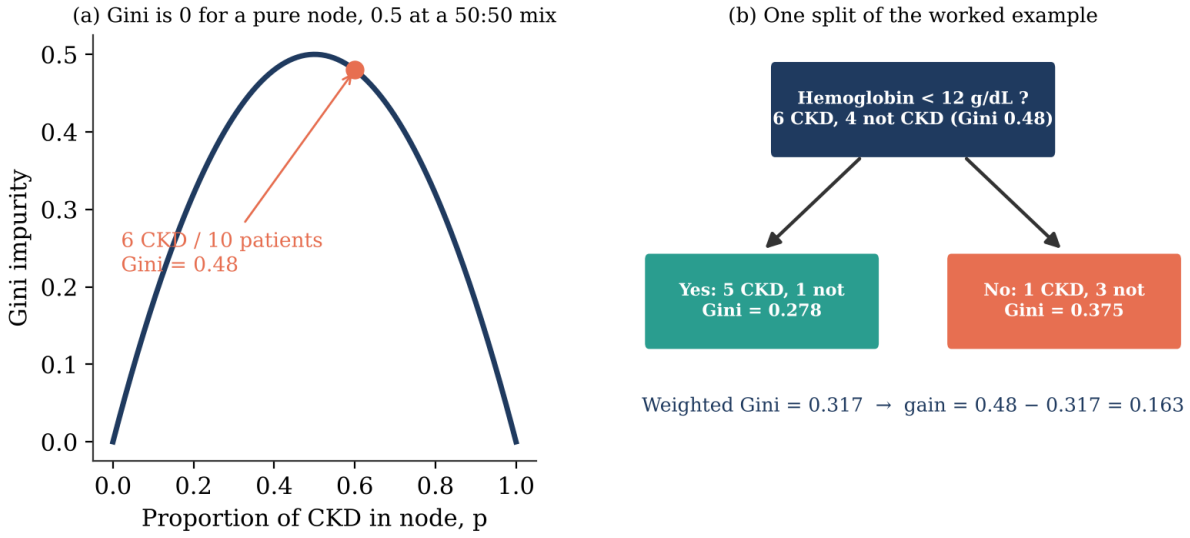


Fig. 11 *(a) Gini impurity for two classes. (b) One split from the worked example.*

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)

A node has 10 patients: 6 CKD and 4 not CKD. The question is "hemoglobin < 12 g/dl?".

Parent: Gini = $1 - (0.6^2 + 0.4^2) = 1 - 0.52 =$ **0.48**.

Yes branch (6 patients: 5 CKD, 1 not): Gini = $1 - ((5/6)^2 + (1/6)^2) =$ **0.278**. No branch (4 patients: 1 CKD, 3 not): Gini = $1 - ((1/4)^2 + (3/4)^2) =$ **0.375**.

Weighted Gini = (6/10)(0.278) + (4/10)(0.375) = 0.317. Gain = 0.48 − 0.317 = **0.163**. The tree compares this gain with other candidate questions and keeps the best.

Strengths and limitations: Very interpretable and handles mixed data types without scaling. A single tree is unstable and prone to overfitting, as small changes in data can change the tree.

4.4.5 Naive Bayes (NB)

IN SIMPLE WORDS

Start from how common CKD is, then multiply by how typical each finding is for CKD patients and for healthy patients. The side with the larger product wins.

Naive Bayes applies Bayes' theorem with the assumption that features are conditionally independent given the class. For continuous attributes, the Gaussian version models each feature with a normal distribution per class. The class with the highest posterior probability is selected.

Step by step

1. Compute the prior probability of each class from the training data (250/400 = 0.625 for CKD and 150/400 = 0.375 for not CKD).
2. For each feature and each class, estimate the mean and standard deviation (numeric features) or the category frequencies (nominal features).
3. For a new patient, compute the likelihood of each observed value under each class (Fig. 12).
4. Multiply the prior by all the likelihoods. The “naive” assumption is that this simple product is allowed because features are treated as independent.
5. Choose the class with the larger result. Dividing by the sum of both results gives the posterior probability.

Formulas

$$P(C \mid \mathbf{x}) = \frac{P(C)\prod_i P(x_i \mid C)}{P(\mathbf{x})} \tag{18}$$

$$P(x_i \mid C) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x_i-\mu)^2}{2\sigma^2}\right) \tag{19}$$

$$\hat{y} = \operatorname{argmax}_C \; P(C)\prod_i P(x_i \mid C) \tag{20}$$

P(C) is the prior of class C and $P(x_i \mid C)$ is the likelihood of feature value x_i in class C. In the Gaussian formula, μ and σ are the mean and standard deviation of that feature computed inside class C. The denominator P(x) is the same for both classes, so it can be ignored when only the winner is needed.

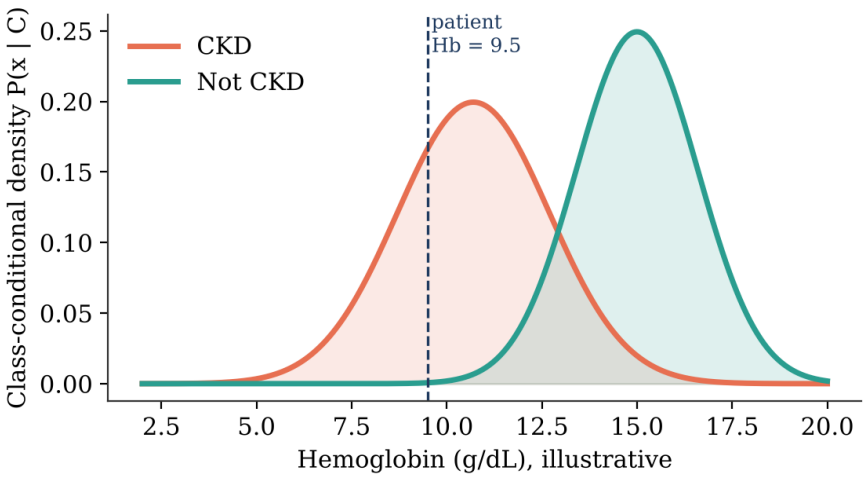


Fig. 12 *Gaussian likelihoods of hemoglobin for the two classes (illustrative curves). A patient with 9.5 g/dl is far more typical of the CKD class.*

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)

A patient has hypertension, anemia and a raised urine albumin level. Assume the likelihoods below, estimated from training data.

Quantity	CKD	Not CKD	Meaning
Prior P(C)	0.625	0.375	Share of each class
P(hypertension | C)	0.50	0.02	Finding is common in CKD
P(anemia | C)	0.40	0.05	
P(raised albumin | C)	0.80	0.05	
Product	**0.1000**	**0.0000188**	Prior × likelihoods

Posterior for CKD = 0.1000 / (0.1000 + 0.0000188) = **0.9998**, so the patient is classified as **CKD**.

Strengths and limitations: Extremely fast and works with little data. The independence assumption is unrealistic for correlated lab values such as hemoglobin and packed cell volume, which limits accuracy.

4.4.6 Random Forest (RF) [6]

IN SIMPLE WORDS

Ask a large committee of slightly different decision trees and go with the majority. Individual trees make different mistakes, so the mistakes cancel out.

Random Forest is a bagging ensemble. It builds many decision trees, each trained on a bootstrap sample of the data and using a random subset of features at every split. The final prediction is the majority vote of all trees. Randomness reduces the variance of individual trees, making the forest robust to noise and missing-value imputation. It also provides feature importance scores, which were used in Section 4.3.

Step by step

1. **Bootstrap.** Draw n = 280 patients from the training set *with replacement* to make a new training sample. Some patients appear twice, some not at all.
2. **Grow a tree** on this sample. At every split, only a random subset of features (the square root of the number of features, about 4 of 24) is allowed to compete.
3. **Repeat** steps 1 and 2 until N = 100 different trees exist (Fig. 13).
4. **Vote.** Each tree predicts a class for the new patient. The class with the most votes is the final answer, and the vote share can be read as a probability.

Formulas

$$\hat{y}(\mathbf{x}) = \text{mode}\{T_1(\mathbf{x}),\ T_2(\mathbf{x}),\ \ldots,\ T_N(\mathbf{x})\}, \qquad N = 100 \tag{21}$$

$$P(\text{left out}) = \left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.37 \tag{22}$$

$$\text{Var} = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{N} \tag{23}$$

$T_\beta(x)$ is the prediction of the b-th tree. Equation (22) shows that each bootstrap sample leaves out about 37% of the patients; these “out-of-bag” patients give a free internal test. In Equation (23), σ^2 is the variance of one tree, ρ

is the correlation between trees and N is the number of trees. The second term shrinks as N grows, and random feature selection keeps ρ low. That is why a forest is more stable than one tree.

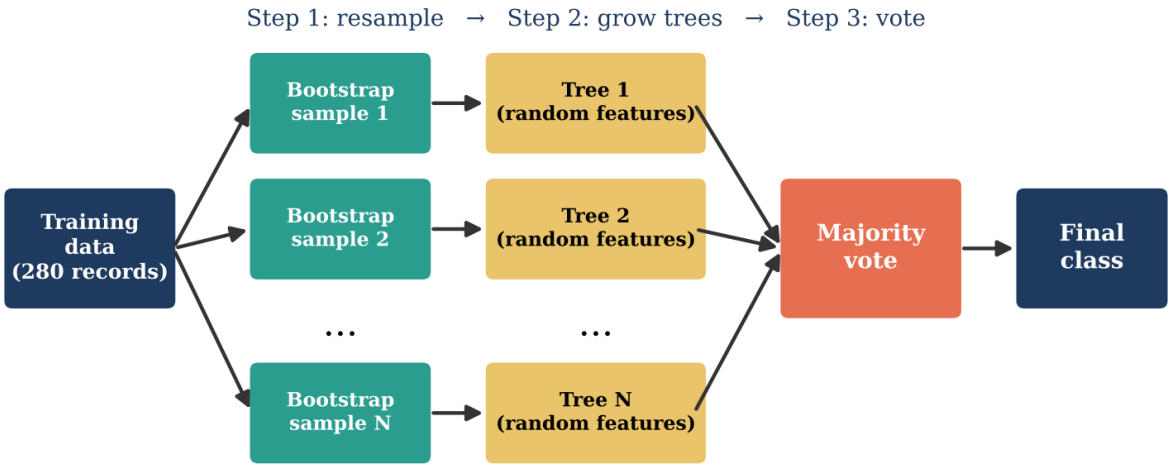


Fig. 13 *Random Forest: bootstrap samples are drawn, one tree is grown on each, and the trees vote.*

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)

A new patient is passed to all 100 trees: 94 trees say CKD and 6 say not CKD. The majority vote is **CKD**, with a vote share of 0.94.

Variance effect: if one tree has variance $\sigma^2 = 1$ and the correlation between trees is $\rho = 0.3$, the average of N = 100 trees has variance $0.3 + 0.7/100 =$ **0.307**, about three times lower than a single tree.

Out-of-bag share for n = 280: $(1 - 1/280)^{280} =$ **0.367**, so about 37% of patients are unseen by each tree.

Strengths and limitations: High accuracy, resistant to overfitting and gives feature importance. It is less interpretable than a single tree and needs more memory.

4.4.7 Gradient Boosting (GB) [7], [8]

IN SIMPLE WORDS

Build the model in rounds. Each new small tree studies where the current model is still wrong and adds a small correction.

Gradient Boosting builds trees one after another. Each new shallow tree is trained to correct the residual errors of the current ensemble, and its contribution is scaled by a learning rate. This stage-wise process gradually reduces the loss function and typically gives very accurate models.

Step by step

1. **Start** with a constant prediction F_0: the log-odds of CKD in the training data.
2. **Residuals.** For every training patient, compute the residual $r = y - p$, the gap between the true label (1 or 0) and the currently predicted probability.
3. **Fit a small tree** h_m (depth 3) that predicts these residuals from the features.
4. **Update** the model by adding a small step of the new tree: $F_m = F_{m-1} + \nu \cdot h_m$, where $\nu = 0.1$ is the learning rate.
5. **Repeat** for 150 rounds (Fig. 14). Convert the final score to a probability with the sigmoid function.

Formulas

$$F_0 = \ln\left(\frac{p}{1-p}\right) \tag{24}$$

$$r_i = y_i - \sigma(F_{m-1}(\mathbf{x}_i)) \tag{25}$$

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu\, h_m(\mathbf{x}) \tag{26}$$

$$P(\text{CKD} \mid \mathbf{x}) = \frac{1}{1 + e^{-F_M(\mathbf{x})}} \tag{27}$$

p is the share of CKD patients, σ is the sigmoid function from Equation (7), and r_i is the pseudo-residual, which equals the negative gradient of the log-loss. Adding trees that follow this gradient is a form of gradient descent in function space, hence the name.

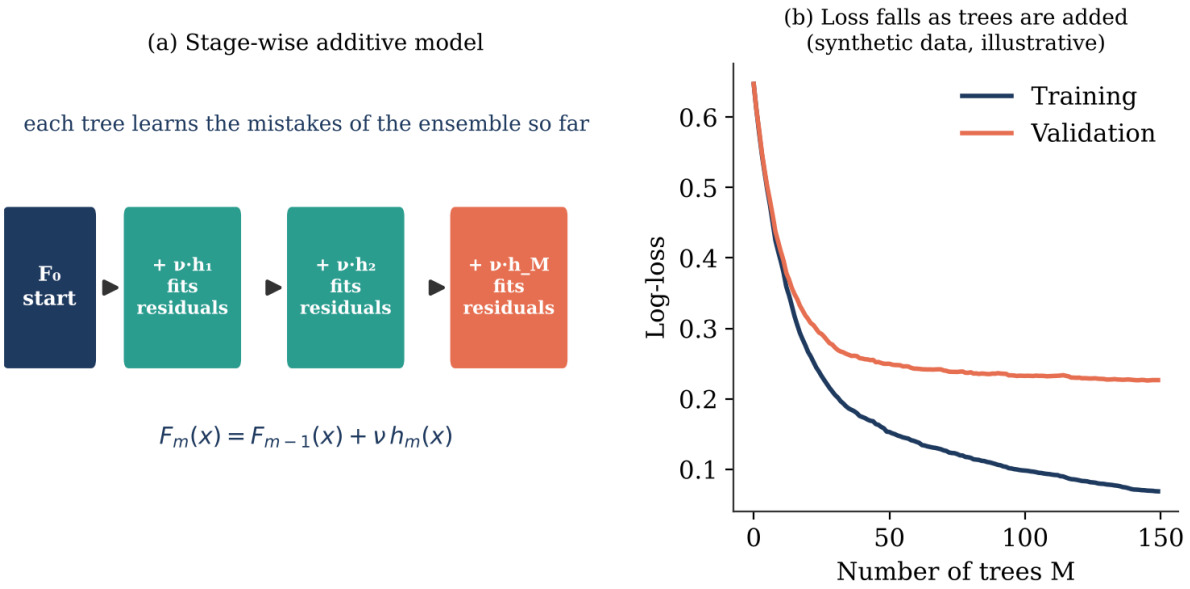


Fig. 14 *(a) Gradient Boosting adds small trees one after another. (b) Training and validation loss fall as more trees are added (synthetic data, illustrative).*

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)

The training data has 62.5% CKD patients, so $F_0 = \ln(0.625 / 0.375) =$ **0.511** and every patient starts with p = 0.625.

For a CKD patient (y = 1) the residual is $r = 1 - 0.625 =$ **0.375**.

Suppose the first tree outputs 1.2 for this patient's leaf. With $\nu = 0.1$: $F_1 = 0.511 + 0.1 \times 1.2 =$ **0.631**, so the new probability is $\sigma(0.631) =$ **0.653**.

The residual shrinks from 0.375 to 0.347. After many rounds it moves close to 0 and the probability approaches 1.

Strengths and limitations: Very accurate and flexible. Training is sequential and slower, and it is sensitive to hyperparameters such as learning rate and number of trees.

4.4.8 Other Models Noted in the Literature

Two further model families appear often in CKD research but were not part of this comparison. **XGBoost** [8] is an optimised implementation of gradient boosting that adds built-in regularisation and fast, parallel tree building, and it frequently reaches state-of-the-art results on tabular clinical data. **Artificial Neural Networks**

(ANN) are layers of connected artificial neurons that can learn complex non-linear relationships, and they are most useful on very large or high-dimensional data such as raw electronic health record streams. Both are listed as candidates for future work.

4.5 Hyperparameters

Table 8. Main hyperparameters of the classifiers

Classifier	Key hyperparameters (scikit-learn)
Logistic Regression	penalty = L2, C = 1.0, solver = lbfgs, max_iter = 1000
KNN	n_neighbors = 5, metric = Euclidean, weights = uniform
SVM	kernel = RBF, C = 10, gamma = scale
Decision Tree	criterion = gini, max_depth = 5, min_samples_split = 2
Naive Bayes	Gaussian, var_smoothing = 1e-9
Random Forest	n_estimators = 100, max_features = sqrt, random_state = 42
Gradient Boosting	n_estimators = 150, learning_rate = 0.1, max_depth = 3

4.6 Experimental Setup

The data was split 70:30 into training (280 records) and test (120 records: 75 CKD and 45 not-CKD) sets with stratification, which keeps the class proportions the same in both parts. Hyperparameters were tuned with 5-fold cross-validation on the training set (Fig. 15). Implementation was done in Python using scikit-learn [12] on Google Colab.

How 5-fold cross-validation works

1. Cut the 280 training records into 5 equal folds.
2. Train on 4 folds and measure the score on the remaining fold.
3. Repeat so that every fold is used once for validation.
4. Average the 5 scores. The settings with the best average are kept.

$$\text{CV score} = \frac{1}{5}\sum_{k=1}^{5} \text{score}_k \tag{28}$$

The test set is touched only once, at the very end, so the final results in Section 5 come from patients the models have never seen.

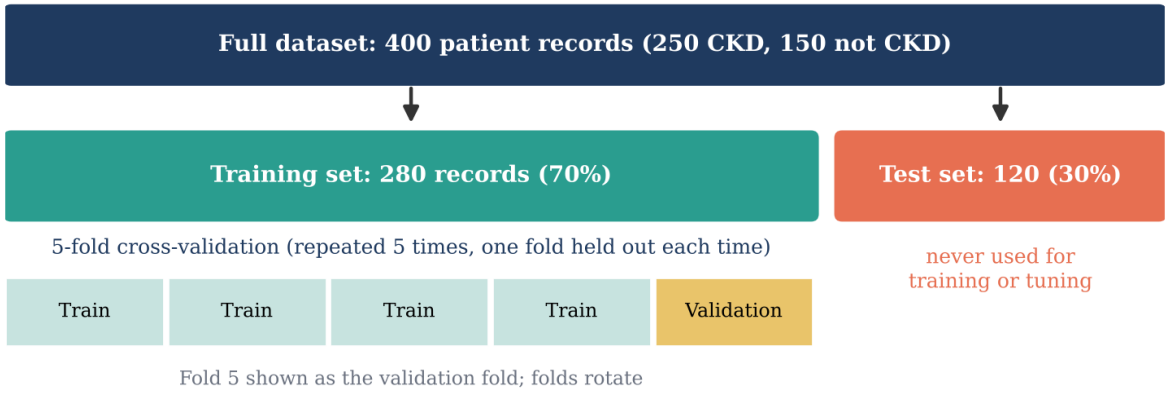


Fig. 15 *Train/test split and 5-fold cross-validation used for tuning.*

4.7 Evaluation Metrics

A confusion matrix counts how many patients were classified correctly and incorrectly (Fig. 16). TP (true positives) are CKD patients correctly found, TN (true negatives) are healthy patients correctly cleared, FP (false positives) are healthy patients wrongly flagged and FN (false negatives) are CKD patients that were missed. All metrics are built from these four counts.

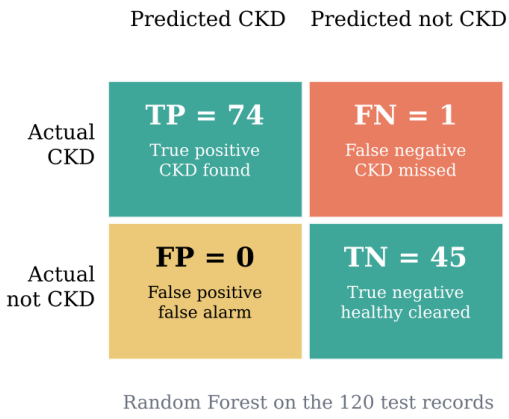


Fig. 16 *Anatomy of a confusion matrix, filled with the Random Forest result on the 120 test patients.*

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{29}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{30}$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{31}$$

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}, \quad \text{AUC} = \int_0^1 \text{TPR}\ d(\text{FPR}) \tag{32}$$

Accuracy is the share of all patients classified correctly. **Precision** answers: of the patients flagged as CKD, how many really have CKD? **Recall** (sensitivity) answers: of the patients who really have CKD, how many were found? **F1** is a balanced average of the two. The **ROC curve** plots TPR against FPR as the decision threshold moves from 0 to 1, and **AUC** is the area under that curve: 1.0 is perfect and 0.5 is no better than guessing. Equivalently, AUC is the probability that a randomly chosen CKD patient receives a higher score than a randomly chosen healthy patient.

> **WORKED EXAMPLE (ILLUSTRATIVE NUMBERS)**
>
> Random Forest on the test set: TP = 74, FN = 1, FP = 0, TN = 45.
>
> Accuracy = (74 + 45) / 120 = **0.9917**. Precision = 74 / (74 + 0) = **1.000**. Recall = 74 / (74 + 1) = **0.9867**.
>
> F1 = 2 × 1.000 × 0.9867 / (1.000 + 0.9867) = **0.9933**. These match the Random Forest row of Table 9.

5. Results and Discussion

Table 9 summarises the performance of all classifiers on the 120 unseen test records. Fig. 17 to Fig. 19 present the same results graphically, and Fig. 20 compares the four strongest-scoring models on all five measures.

Table 9. Performance comparison of classifiers on the test set (%, AUC as a fraction)

Classifier	Accuracy	Precision	Recall	F1-score	AUC
Logistic Regression	95.83	97.30	96.00	96.64	0.985
KNN	93.33	95.89	93.33	94.59	0.962
SVM	97.50	98.65	97.33	97.99	0.990
Decision Tree	96.67	97.33	97.33	97.33	0.968
Naive Bayes	94.17	94.74	96.00	95.36	0.972
Random Forest	**99.17**	**100.00**	**98.67**	**99.33**	**0.999**
Gradient Boosting	98.33	98.67	98.67	98.67	0.995

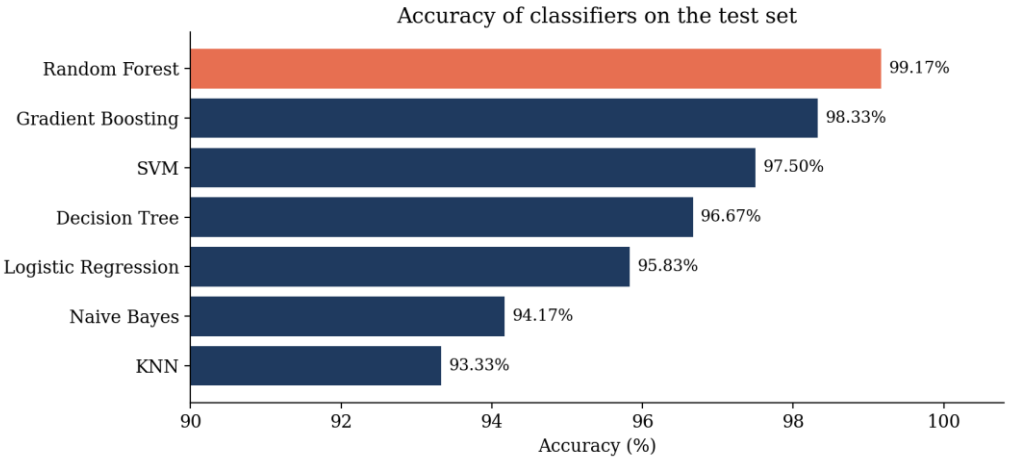


Fig. 17 *Accuracy comparison of the classifiers.*

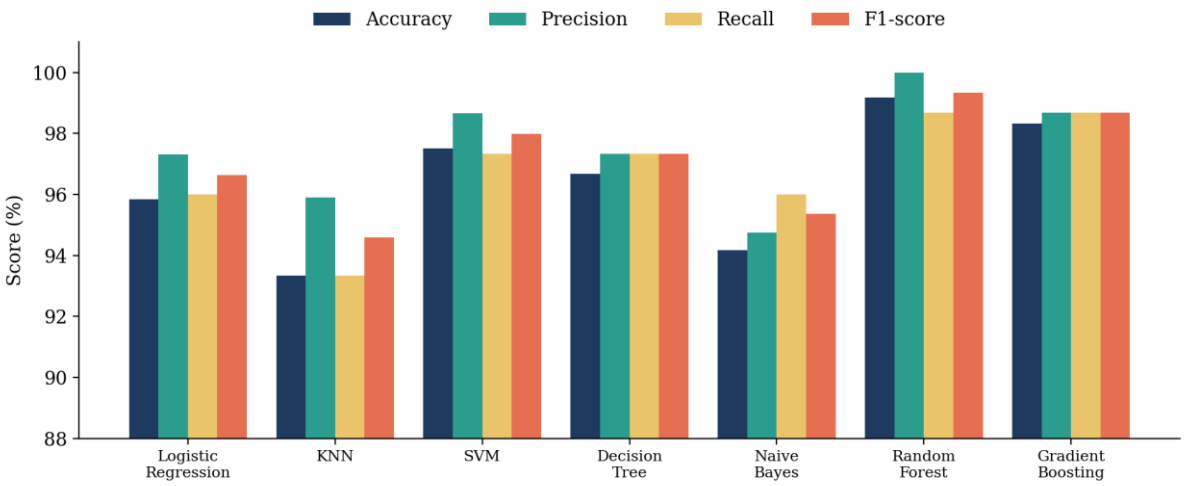


Fig. 18 *Accuracy, precision, recall and F1-score of each classifier.*

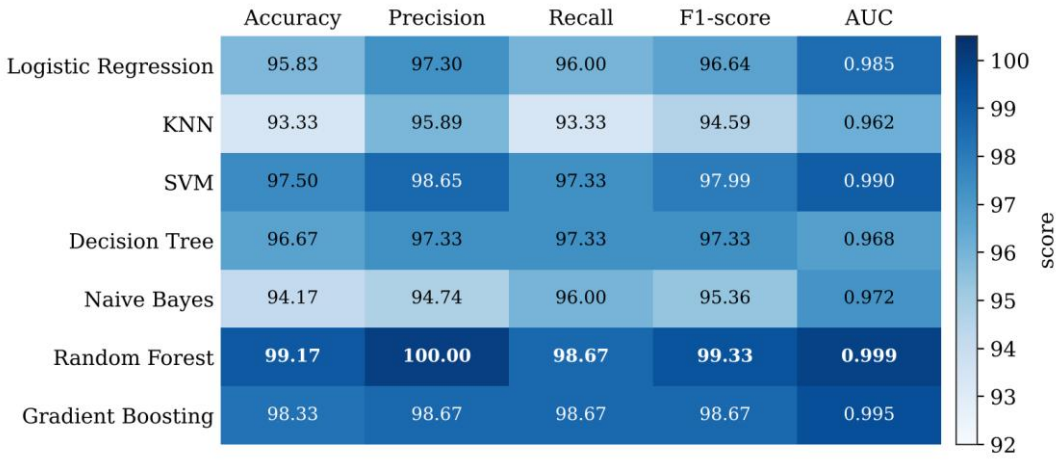


Fig. 19 *Heat map of all measures. Darker cells mean better scores; Random Forest is darkest in every column.*

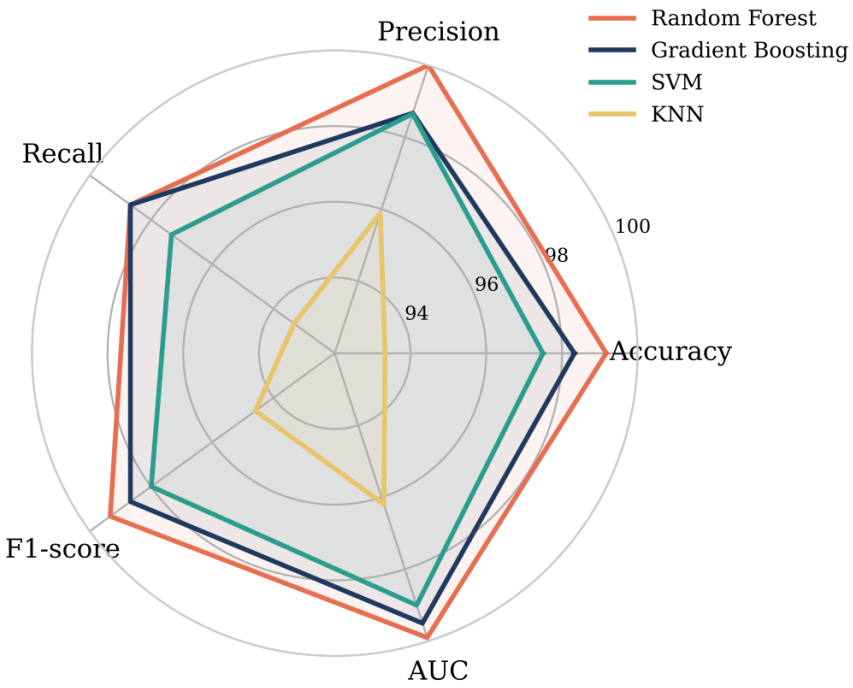


Fig. 20 *Radar chart of Random Forest, Gradient Boosting, SVM and KNN (axis starts at 92%).*

5.1 Analysis

Random Forest delivered the best overall performance (accuracy 99.17%, F1-score 99.33%), misclassifying only one of the 120 test patients. Gradient Boosting (98.33%, two errors) and SVM (97.50%, three errors) followed closely. Decision Tree (four errors) and Logistic Regression (five errors) performed well but were less stable, while KNN gave the lowest accuracy (93.33%, eight errors), probably because distance calculations are sensitive to noise and imputed values. Naive Bayes (seven errors) was fast but its independence assumption is violated by correlated lab values such as hemoglobin and packed cell volume.

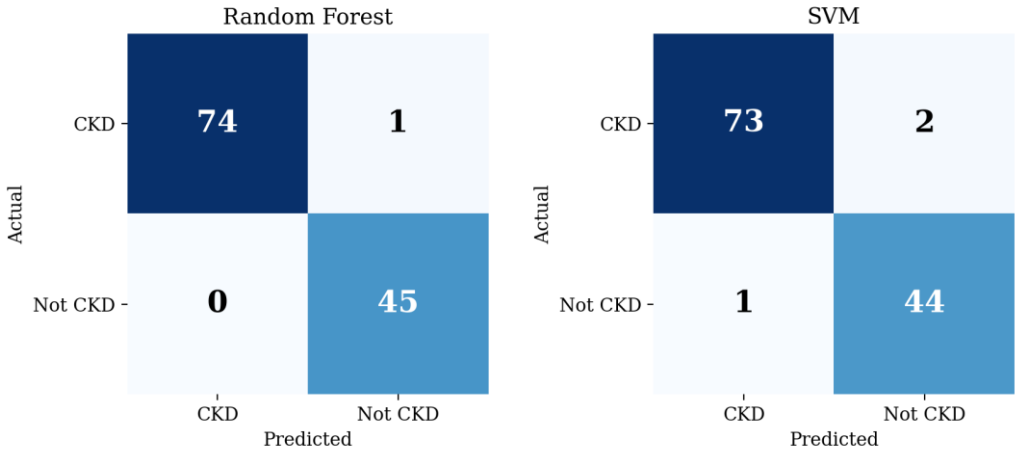


Fig. 21 *Confusion matrices of Random Forest and SVM.*

The confusion matrices in Fig. 21 show that Random Forest produced no false positives and a single false negative. In a medical setting, false negatives are the more costly error because a diseased patient is missed, so the high recall of Random Forest (98.67%) is particularly valuable.

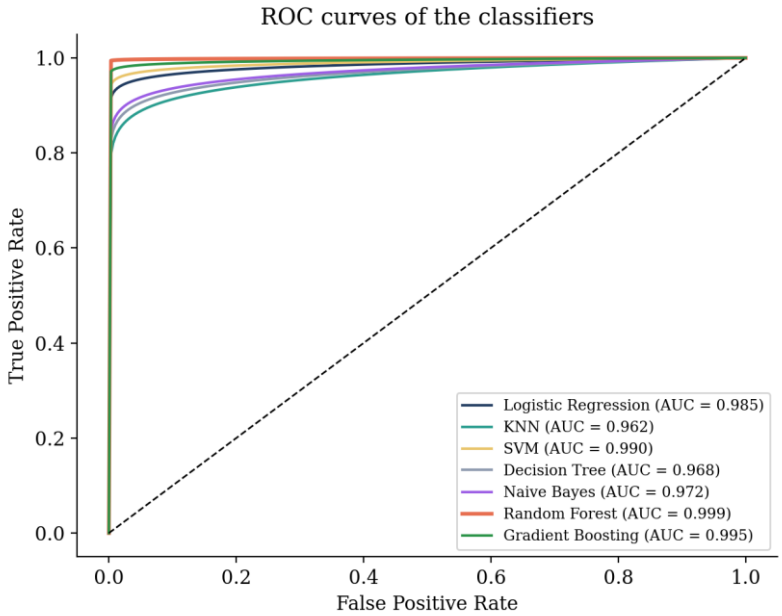


Fig. 22 *ROC curves of all classifiers.*

The ROC curves in Fig. 22 confirm this ranking: the Random Forest curve hugs the top-left corner with an AUC of 0.999, while KNN has the lowest AUC (0.962). All classifiers exceeded 0.96, which shows that the selected attributes are strongly discriminative. The important role of hemoglobin, specific gravity, serum creatinine and albumin (Fig. 6) agrees with clinical knowledge, since these markers reflect anemia, urine concentration ability, filtration and protein loss respectively.

5.2 Why Random Forest Performed Best

The model explanations in Section 4.4 point to three reasons:

- **Lower variance.** Averaging many decorrelated trees (Equation (23)) removes the instability that a single Decision Tree shows.
- **Robust to messy data.** Trees do not need scaling and split on thresholds, so imputed values and outliers have less influence than in KNN or Naive Bayes.

- **Non-linear and mixed features.** CKD depends on combinations of lab values and yes/no findings, which trees capture easily but a linear model cannot.

5.3 Limitations

The dataset is small (400 records) and comes from a single source, so results may not generalise to other populations. High accuracy on the UCI data should not be read as clinical validity; external validation on larger, multi-centre data is required before real-world use. The worked examples in Section 4 use simplified numbers to explain the mathematics and are not fitted values from the dataset.

6. Conclusion and Future Work

This paper compared seven machine learning classifiers for CKD prognosis using the UCI CKD dataset. After pre-processing and feature ranking, Random Forest gave the best results with 99.17% accuracy and an AUC of 0.999, ahead of Gradient Boosting and SVM. The study shows that ML models built on routine clinical parameters can support early identification of CKD patients.

In future work we plan to:

1. use larger, multi-centre datasets;
2. predict CKD stages and progression rather than only presence;
3. apply deep learning and explainability tools such as SHAP; and
4. deploy the best model as a web-based screening application.

References

[1] A. S. Levey and J. Coresh, “Chronic kidney disease,” The Lancet, vol. 379, no. 9811, pp. 165-180, 2012.

[2] N. R. Hill et al., “Global prevalence of chronic kidney disease - a systematic review and meta-analysis,” PLoS ONE, vol. 11, no. 7, e0158765, 2016.

[3] A. Salekin and J. Stankovic, “Detection of chronic kidney disease and selecting important predictive attributes,” in Proc. IEEE Int. Conf. Healthcare Informatics (ICHI), 2016, pp. 262-270.

[4] A. Subasi, E. Alickovic, and J. Kevric, “Diagnosis of chronic kidney disease by using random forest,” in Proc. CMBEBIH, Springer, 2017.

[5] A. Almansour et al., “Neural network and support vector machine for the prediction of chronic kidney disease: A comparative study,” Computers in Biology and Medicine, vol. 109, pp. 101-111, 2019.

[6] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[7] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.

[8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, 2016, pp. 785-794.

[9] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273-297, 1995.

[10] T. Cover and P. Hart, “Nearest neighbor pattern classification,” IEEE Trans. Information Theory, vol. 13, no. 1, pp. 21-27, 1967.

[11] L. J. Rubini, P. Soundarapandian, and P. Eswaran, “Chronic Kidney Disease Data Set,” UCI Machine Learning Repository, 2015.

[12] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[13] KDIGO CKD Work Group, “KDIGO 2012 clinical practice guideline for the evaluation and management of chronic kidney disease,” Kidney Int. Suppl., vol. 3, no. 1, pp. 1-150, 2013.

[14] B. Bikbov et al., “Global, regional, and national burden of chronic kidney disease, 1990-2017: a systematic analysis for the Global Burden of Disease Study 2017,” The Lancet, vol. 395, no. 10225, pp. 709-733, 2020.

[15] L. A. Inker et al., “New creatinine- and cystatin C-based equations to estimate GFR without race,” N. Engl. J. Med., vol. 385, no. 19, pp. 1737-1749, 2021.

[16] H. Polat, H. Danaei Mehr, and A. Cetin, “Diagnosis of chronic kidney disease based on support vector machine by feature selection methods,” J. Medical Systems, vol. 41, no. 4, art. 55, 2017.

[17] N. Tazin, S. A. Sabab, and M. T. Chowdhury, “Diagnosis of chronic kidney disease using effective classification and feature selection technique,” in Proc. Int. Conf. Medical Engineering, Health Informatics and Technology (MediTec), 2016.

[18] J. Qin et al., “A machine learning methodology for diagnosing chronic kidney disease,” IEEE Access, vol. 8, pp. 20991-21002, 2020.